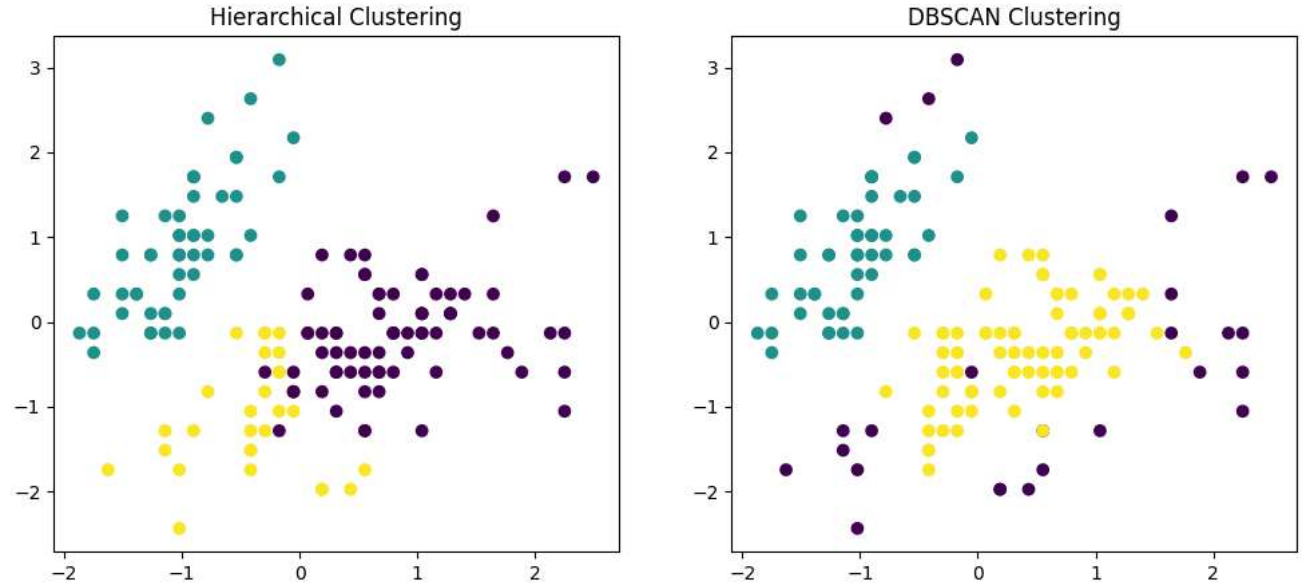
Practical 6: Clustering Algorithms## Practical Information| Attribute | Details ||---------|---------|| **Practical Number** | 6 || **Title** | Clustering Algorithms || **Duration** | 2 Hours || **Course Outcomes (COs)** | CO3, CO4 || **Description** | Implement various clustering algorithms and evaluate cluster quality. |## Key Topics Covered- K-Means- Hierarchical Clustering- DBSCAN- GMM- Silhouette analysis---## Instructions for Students1. **Complete all tasks** outlined in this practical2. **Write your code** in the cells provided below3. **Add comments** to explain your code4. **Include proper output** (prints, visualizations, etc.)5. **Submit your completed notebook** before the deadline---## Your Work Starts HereUse the cell(s) below to write your implementation:

```
# Practical 6: Clustering Algorithms
# Write your implementation here

# Practical 6: Clustering Algorithms - Hierarchical & DBSCAN

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.cluster import AgglomerativeClustering, DBSCAN
from sklearn.preprocessing import StandardScaler

# Load dataset
data = load_iris()
X = data.data

# Feature scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Hierarchical Clustering
hierarchical = AgglomerativeClustering(n_clusters=3)
hier_labels = hierarchical.fit_predict(X_scaled)

print("Hierarchical Clustering: Number of clusters =", len(set(hier_labels)))

# DBSCAN
dbscan = DBSCAN(eps=0.6, min_samples=5)
db_labels = dbscan.fit_predict(X_scaled)

clusters = len(set(db_labels)) - (1 if -1 in db_labels else 0)
outliers = list(db_labels).count(-1)

print("DBSCAN: Clusters =", clusters)
print("DBSCAN: Outliers =", outliers)

# Visualization
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=hier_labels)
plt.title("Hierarchical Clustering")

plt.subplot(1, 2, 2)
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=db_labels)
plt.title("DBSCAN Clustering")

plt.show()
```

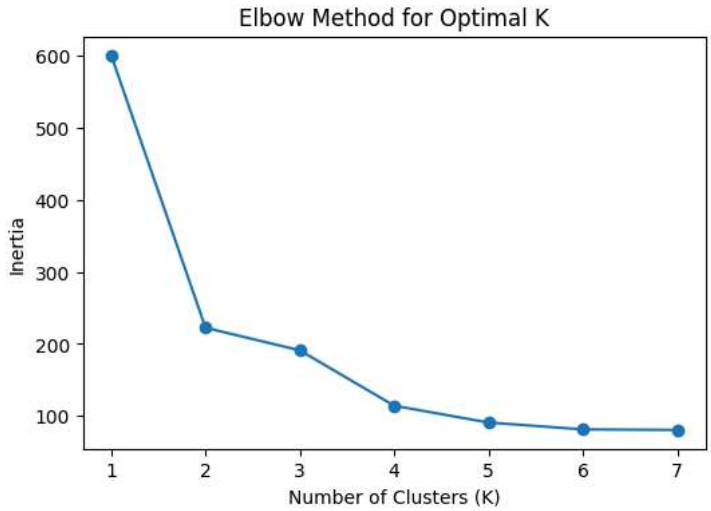
```
Hierarchical Clustering: Number of clusters = 3
DBSCAN: Clusters = 2
DBSCAN: Outliers = 26
```



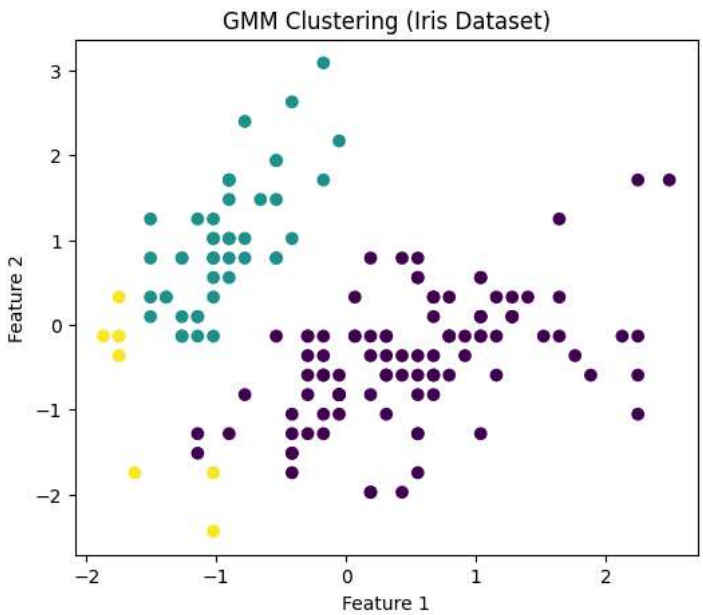

```
# Additional code cell

# Practical 6: Clustering Algorithms - GMM + Optimal K

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.mixture import GaussianMixture
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# Load dataset
data = load_iris()
X = data.data

# Feature scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# -------------------------
# Find Optimal K (Elbow Method)
# -------------------------
inertia_values = []

K_range = range(1, 8)
for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_scaled)
    inertia_values.append(kmeans.inertia_)

print("Elbow method completed.")

plt.figure(figsize=(6, 4))
plt.plot(K_range, inertia_values, marker='o')
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia")
plt.title("Elbow Method for Optimal K")
plt.show()

# -------------------------
# Gaussian Mixture Model (GMM)
# -------------------------
gmm = GaussianMixture(n_components=3, random_state=42)
gmm_labels = gmm.fit_predict(X_scaled)

print("GMM: Number of clusters = 3")

# Visualization
```

```
plt.figure(figsize=(6, 5))
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=gmm_labels)
plt.title("GMM Clustering (Iris Dataset)")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

```
Elbow method completed.
```



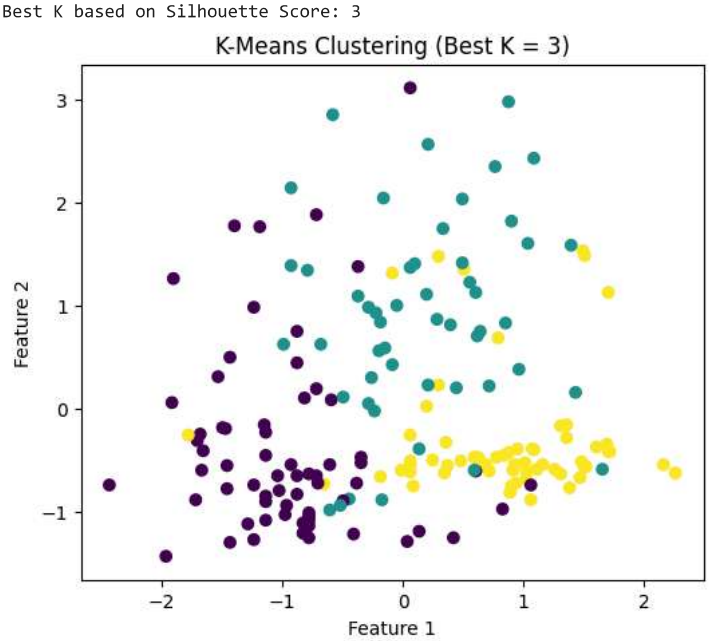

```
GMM: Number of clusters = 3
```




```
# Additional code cell

# Practical 6: Clustering Algorithms - K-Means + Silhouette Analysis

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

# Load dataset
data = load_wine()
X = data.data

# Feature scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# -------------------------
# Find Best K using Silhouette Score
```

```
# Find Best K using Silhouette Score
# -------------------------
sil_scores = []
K_range = range(2, 8)

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    labels = kmeans.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)
    sil_scores.append(score)
    print(f"K = {k}, Silhouette Score = {score:.4f}")

best_k = K_range[np.argmax(sil_scores)]
print(f"\nBest K based on Silhouette Score: {best_k}")

# -------------------------
# Train Final K-Means Model
# -------------------------
kmeans_final = KMeans(n_clusters=best_k, random_state=42)
final_labels = kmeans_final.fit_predict(X_scaled)

# Visualization (first two features)
plt.figure(figsize=(6, 5))
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=final_labels)
plt.title(f"K-Means Clustering (Best K = {best_k})")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()

print("\nClustering completed successfully!")
```

```
K = 2, Silhouette Score = 0.2650
K = 3, Silhouette Score = 0.2849
K = 4, Silhouette Score = 0.2542
K = 5, Silhouette Score = 0.1836
K = 6, Silhouette Score = 0.1690
K = 7, Silhouette Score = 0.1726

Best K based on Silhouette Score: 3
```




```
Clustering completed successfully!
```